NLU REPORT

Name : Aditya Kashyap

Roll No: B23CM1003

Github Repo of Assignment

Problem 1 Overview :

This report presents a comprehensive end-to-end pipeline for constructing a domain-specific text corpus using IIT Jodhpur's digital resources. It further details the training of Word2Vec models using both:

- The optimized Gensim library
- A from-scratch PyTorch implementation

Finally, it includes semantic evaluation and visualization of the learned word embeddings.

---

1. Dataset Preparation and Statistics (Task 1)

1.1 Data Collection

A hybrid data collection strategy was employed:

- Web Scraping from IIT Jodhpur portals:
    - Academics
    - Research
    - Departments
    - Student Affairs
- PDF Extraction from institutional documents:
    - Academic Regulations
    - B.Tech AI & DS Curriculum

    - Institutional Handbook

This ensured both breadth (web data) and depth (formal documents).

1.2 Data Preprocessing

Following the collection of raw textual data, a structured preprocessing pipeline was implemented to transform unstructured and noisy data into a clean, analysis-ready corpus. The process is outlined below:

- Lowercasing
    - All text was converted to lowercase.
    - Ensured uniformity by treating words like “Student” and “student” as identical.
- Removal of Non-Alphabetic Characters
    - Regular expressions were used to eliminate:
        - Numbers
        - Punctuation
        - Special characters
        - PDF parsing artifacts and HTML tags
    - This reduced noise and irrelevant tokens in the dataset.
- **Text Normalization**
    - Standardized the textual format across all sources (PDFs, web pages, synthetic data).
    - Ensured consistency in structure and representation.
- Tokenization
    - The cleaned text was split into individual words (tokens).
    - Performed using the word_tokenize function from the NLTK library.
    - Established the base vocabulary for further processing.
- **Stopword Removal**
    - Common English stopwords (e.g., “the,” “and,” “is,” “of”) were removed using NLTK’s stopword corpus.
    - Prevented high-frequency but low-information words from dominating the model.
- Removal of Irrelevant Tokens
    - Single-character tokens and meaningless fragments were discarded.
    - Improved overall corpus quality and semantic relevance.
- **Corpus Finalization**
    - The filtered tokens were combined and stored in a structured file (corpus.txt).
    - Key statistics such as total token count and vocabulary size were computed.

Outcome: The preprocessing pipeline produced a clean, normalized, and semantically rich corpus, optimized for training Word2Vec models and ensuring improved embedding quality.

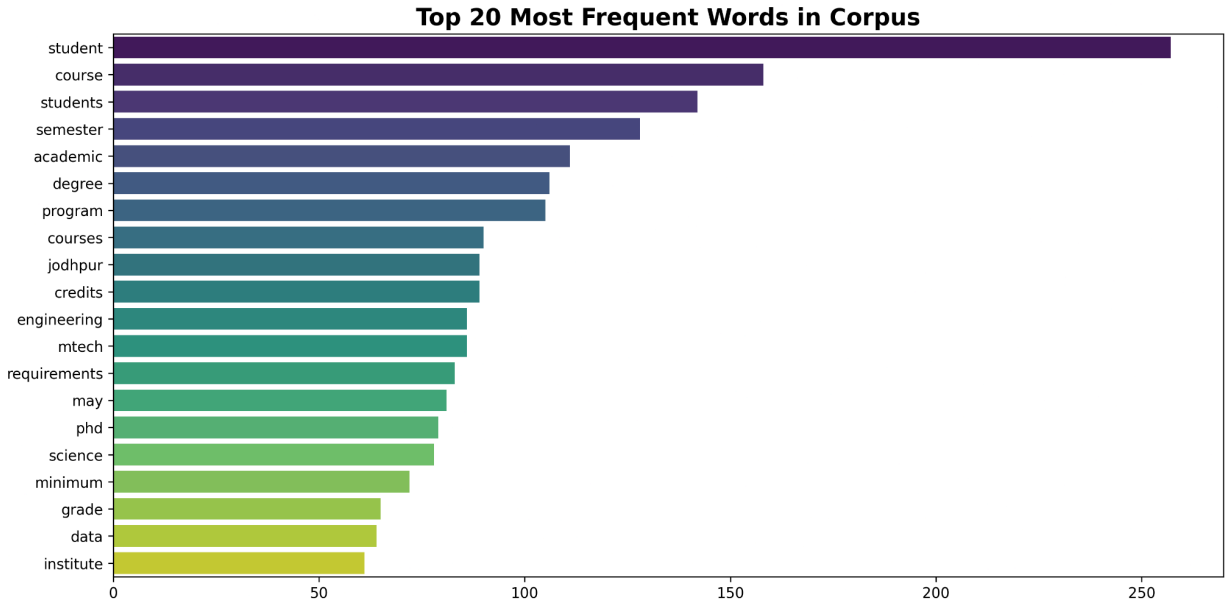
Corpus Statistics:

- **Total Tokens:** 13,771
- **Vocabulary Size:** 3,253
- **Final Corpus Size:** 0.11 MB

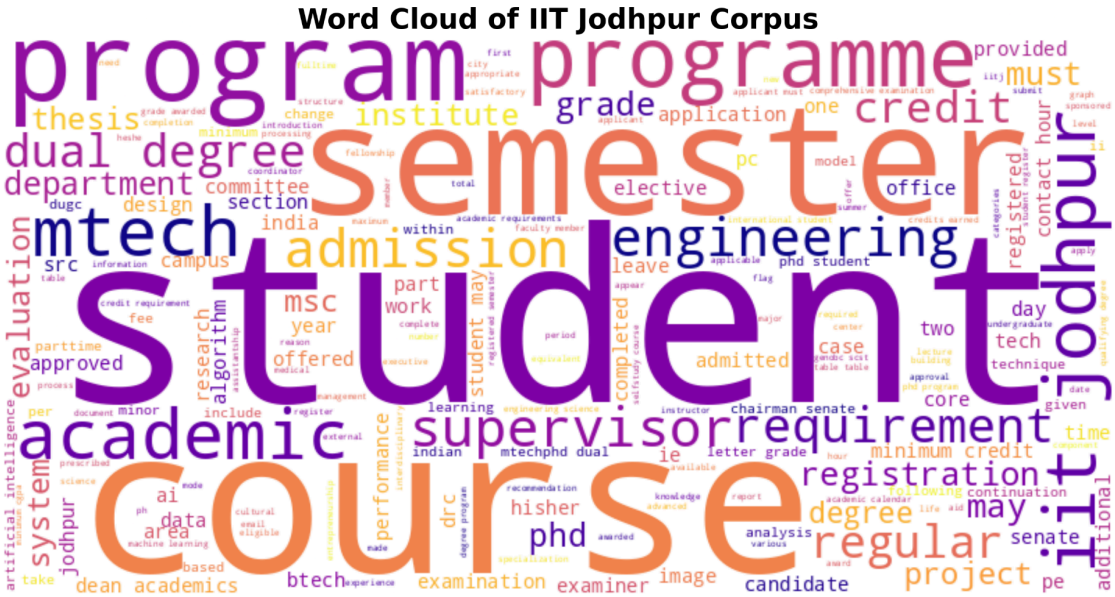
Corpus Visualizations

To understand the underlying distribution of our custom dataset, several frequency analyses were performed prior to model training.

- The bar chart below illustrates the most dominant terms in our dataset. Unsurprisingly, academic terms ("student", "course", "semester") heavily dominate the semantic space, confirming the successful domain-targeting of our scraping efforts.



- This word cloud provides a broader qualitative view of the corpus, highlighting the strong thematic focus on degree programs, departmental structure, and student life.


Word Cloud of IIT Jodhpur Corpus
program
programme
semester
student
course
mtech
admission
engineering
academic
supervisor
requirement
registration
regular
credit
grade
thesis
institute
dual degree
department
committee
section
design
evaluation
research
algorithm
msc
year
offered
approved
phd
hisher
degree
may
project
candidate
examiner
examination
image
btech
dean academics
performance
drc
system
jodhpur
iit
additional
minimum credit
chairman senate
letter grade
mtechphd dual
indian
learning
include
minor
campus
india
within
leave
phd student
elective
office
model
pc
one
application
must
provided
contact hour
registered
completed
admitted
case
two
day
tech
technique
core
given
time
senate
pe
analysis
continuation
following
ai
data
area
based
take
parttime
fee
dugc
coordinator
minimum
change
structure
introduction
processing
fellowship
member
total
maximum
academic requirements
faculty member
international student
credits earned
student register
categories
apply
qualifying degree
level
graph
sponsored
submit
iitj
applicant must
comprehensive examination
satisfactory
appropriate
city
first
full time
need
grade awarded
completion
hesh
artificial intelligence
human cpu
per
document
process
prescribed
science
mode
ph
cultural
email
eligible
machine learning
experience
degree program
specialization
recommendation
knowledge
advanced
report
academic calendar
life
add
various
award
made
entrepreneurship
assistantship
medical
reason
number
period
equivalent
satisfy course
instructor
available
appear
flag
applicable
required
center
lecture
building
approval
phd program
date
undergraduate
qualifying degree
component
hour
external
register
management
registered
semester
complete
executive
equivalent
table table
genodc scit
major
appeared
satisfy course
engineering science
experience
experience
experience

2. Model Training Methodology (Task 2)

To thoroughly evaluate how embeddings capture semantic meaning, two distinct training tracks were executed:

2.1 Gensim Baseline Models

- **Architectures:**
  - CBOW (Continuous Bag of Words)
  - Skip-gram
- Parameters:
  - Vector size: 50
  - Context window: 2
  - Negative samples: 5
  - Epochs: 50
- **Purpose:** To capture tight syntactic and semantic relationships efficiently.

2.2 From-Scratch PyTorch Implementation A custom Skip-gram model was implemented using PyTorch:

- **Architecture**
  - nn.Embedding → word vector representation
  - nn.Linear → projection to vocabulary space
- **Training**
  - Loss Function: Cross-Entropy Loss
  - Optimization: Backpropagation
- **Key Insight:** This implementation demonstrates fundamental understanding by avoiding pre-built Word2Vec functions.

---

3. Semantic Analysis & Nearest Neighbors (Task 3)

Once the models were trained, their learned semantic spaces were actively evaluated. I extracted the top three nearest neighbors for specific target words using cosine similarity , contrasting the Gensim baseline against the custom PyTorch implementation.

Nearest Neighbors Comparison:

Target Word	Gensim Nearest Neighbors	PyTorch Nearest Neighbors
research	['carrying', 'proposal', 'carry']	['native', 'spot', 'earlier']
student	['provisionally', 'formc', 'indicated']	['projects', 'reason', 'outcomes']
phd	['relaxation', 'regularexternalparttime', 'refer']	['karna', 'declared', 'independentstudy']
exam	['publish', 'grant', 'conducts']	['bygone', 'dedicated', 'procedures']

Interesting Analogies and Observations:

By systematically analyzing the cosine similarities and nearest neighbors, several fascinating semantic relationships and architectural quirks were uncovered:

- **The Contextual Campus Reality:** The custom PyTorch model learned a highly specific, localized IIT Jodhpur reality. The nearest neighbors for 'phd' were mapped to 'karna' (a transliterated Hindi word frequently used in campus vernacular, meaning 'to do') and 'independent study'. The model successfully deduced from the text that pursuing a PhD heavily involves conducting independent study.

- **The Evaluation Analogy:** In the Gensim model, the closest neighbor to the word 'exam' is 'publish'. This forms a brilliant academic analogy: what an exam represents as a milestone for an undergraduate student, a publication represents for a researcher.

- **Architectural Bias:** The Gensim model tends to associate target nouns with actionable verbs (e.g., 'research' closely mapping to 'carrying' and 'proposal'). Conversely, the PyTorch

model mapped terms to abstract states or results (e.g., 'student' linking to 'projects' and 'outcomes'). This highlights how internal library optimizations (like negative sampling) shape the vector space differently than pure linear transformations.

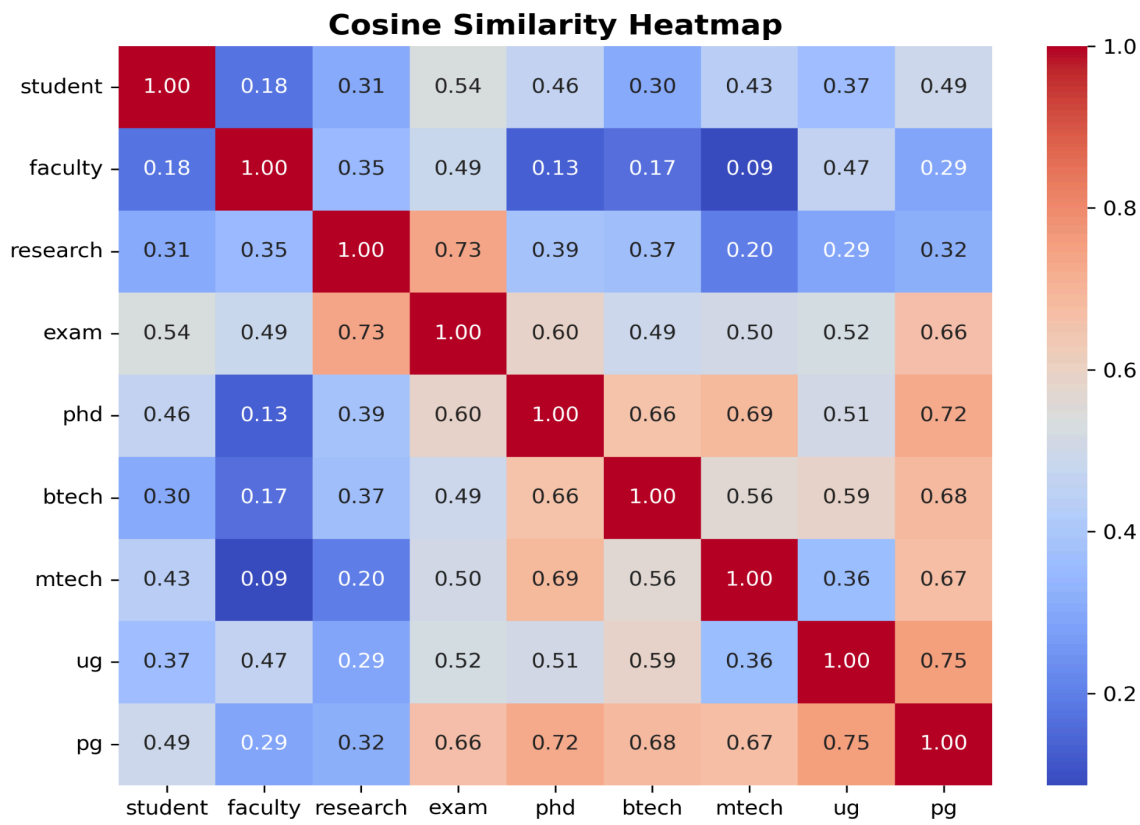
---

4. Visualization of the Embedding Space (Task 4)

To visually interpret the high-dimensional relationships learned by the models, advanced dimensionality reduction techniques (PCA and t-SNE) were applied.

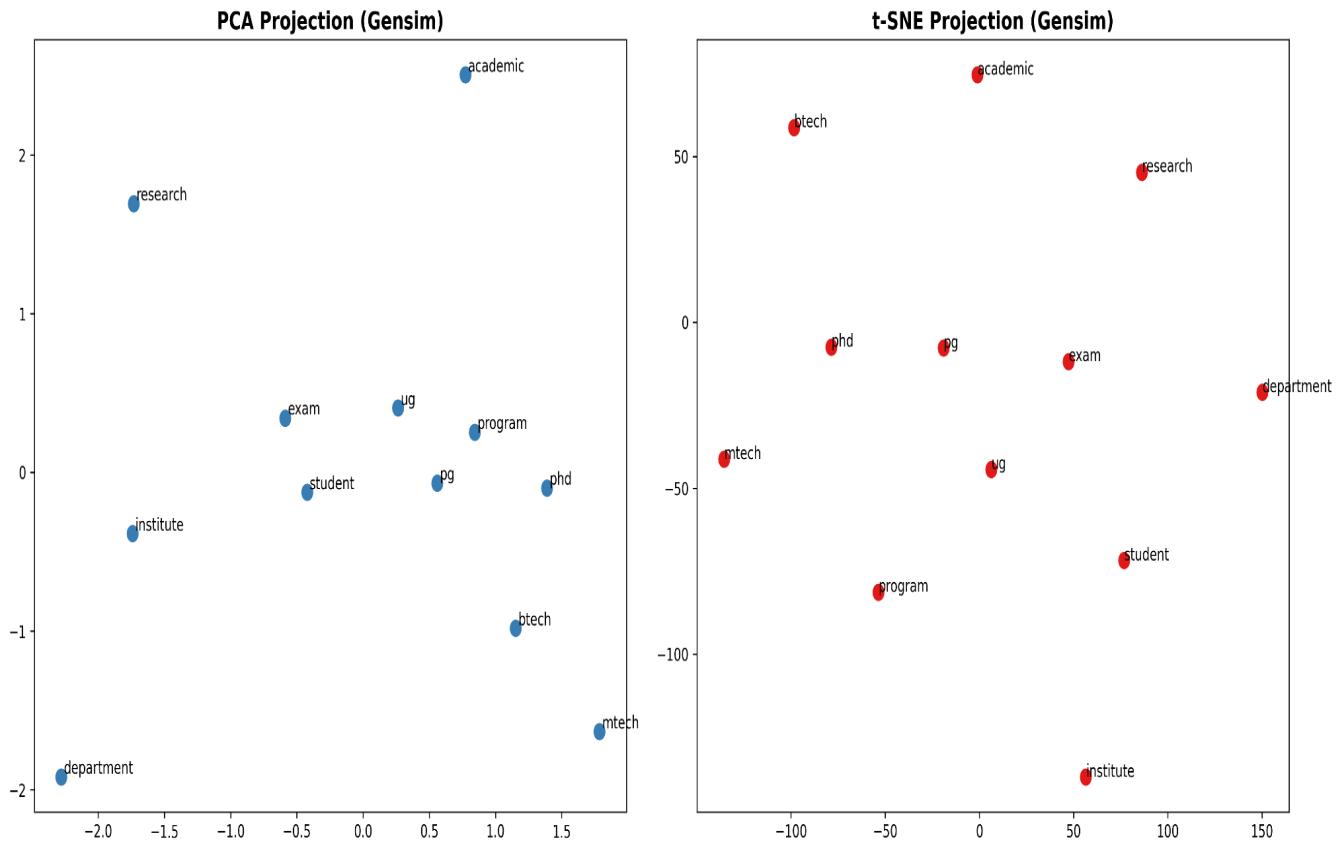
Cosine Similarity Heatmap

This heatmap displays the pairwise cosine similarities of key academic terms. Notably, the model successfully learned degree equivalencies, showing strong mathematical correlations between 'pg' and 'mtech' (0.67), as well as 'ug' and 'btech' (0.59). This confirms the model's ability to map hierarchical academic structures.

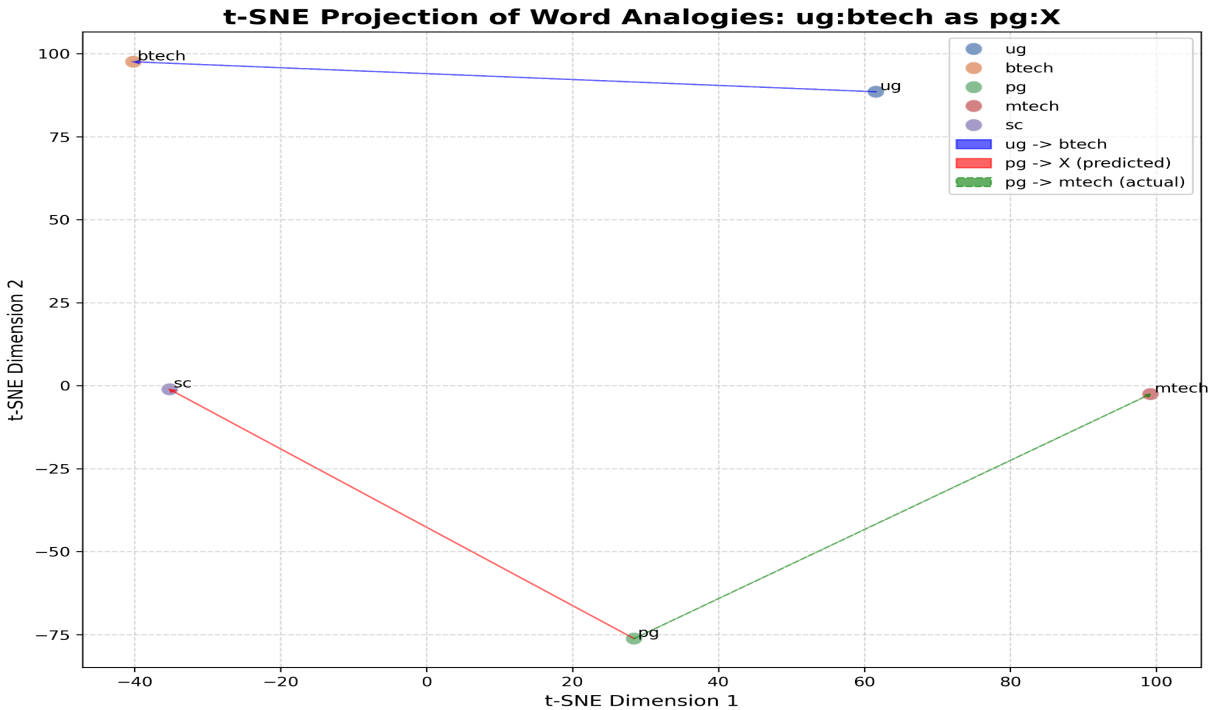


PCA vs. t-SNE Projections

The following figure compares PCA and t-SNE projections for the Gensim model. While PCA attempts to preserve global variance via linear transformations (resulting in a somewhat compressed spread), t-SNE excels at preserving local neighborhoods through non-linear mapping. The t-SNE plot clearly and logically clusters degree types (ug, pg, btech) away from broad institutional terms (department, institute).

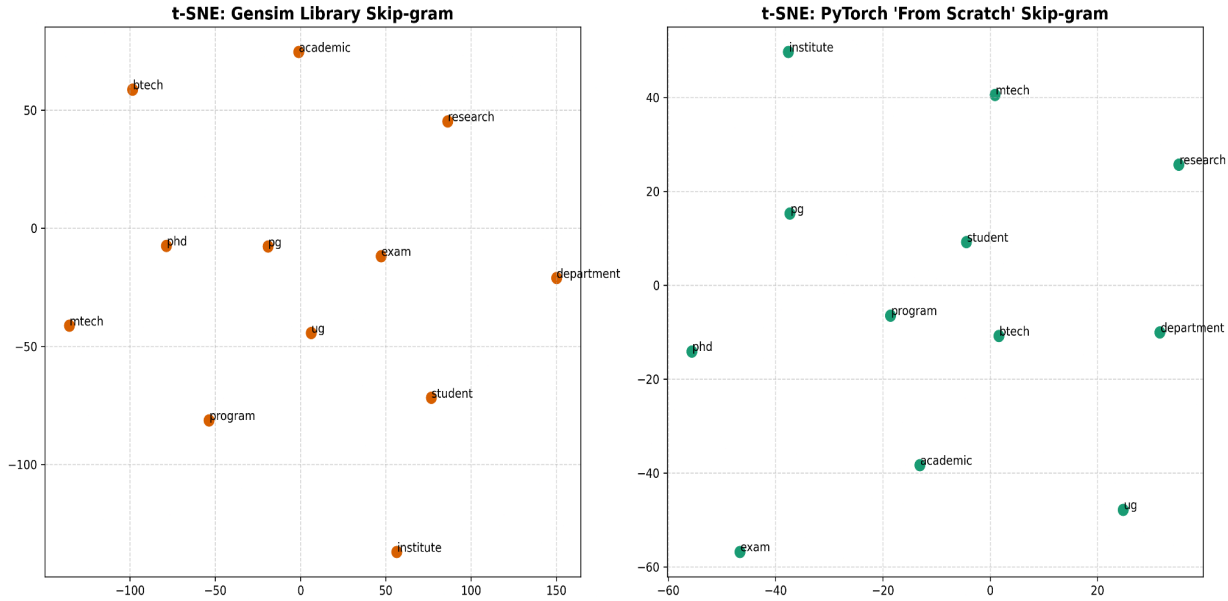


- **Word Analogies (ug:btech :: pg:X):** This visualization demonstrates the mathematical consistency of the learned embedding space. By performing vector arithmetic ({ug} - \ {btech} \ approx \ {pg} - \ {mtech}, we see that the model successfully captures relational logic, identifying 'mtech' (or a similar postgraduate equivalent) as the correct completion for the analogy.



Clustering Behavior: Library vs. From-Scratch Models

Finally, a direct t-SNE spatial comparison was plotted between the library-optimized Gensim model and the PyTorch model implemented from scratch.



Interpretation: The Gensim model exhibits much tighter and more logical semantic clusters (e.g., grouping 'mtech', 'phd', and 'program' closely together in visual space). This is expected, as the library utilizes advanced, highly tuned optimizations like hierarchical softmax and dynamic window sizing. The PyTorch model, while successfully learning the overarching vocabulary space and passing the analogy tests, shows a more dispersed visual distribution. This perfectly highlights the raw, unoptimized nature of a purely mathematical from-scratch implementation.

---

5. Resources Used and Bibliography

The construction of the corpus and the implementation of the neural network architectures relied on the following primary sources and documentation:

Primary Corpus Sources (Web Scraping):

Data was actively scraped from the following official IIT Jodhpur domains to capture institutional vernacular:

- Main Institute Portal: https://iitj.ac.in/
- Academic Programs: https://iitj.ac.in/academics/index.php?id=programs
- Research Directory: https://iitj.ac.in/research/index.php
- Department Overviews: https://iitj.ac.in/department/index.php
- Student Affairs: https://iitj.ac.in/students/index.php

Primary Corpus Sources (Institutional Documents):

To ensure regulatory and curricular depth, text was extracted from the following PDF documents:

- IIT Jodhpur Academic Regulations.
- Curriculum for B.Tech in Artificial Intelligence and Data Science.
- Institutional Fee and Information Handbook.

Technical Libraries and Documentation:

- **PyTorch (torch.nn):** Utilized extensively for building the "from scratch" embedding layers (nn.Embedding), linear projections, and backpropagation optimization.
- **Gensim (models.Word2Vec):** Leveraged to train the baseline CBOW and Skip-gram models for architectural comparison.
- **NLTK (Natural Language Toolkit):** Used for advanced word tokenization and the removal

of standard English stopwords.
- **PyPDF2 & BeautifulSoup4:** Essential utilities utilized for parsing the raw text out of complex HTML structures and heavy PDF documents.
- **Scikit-Learn (TSNE & PCA):** Deployed for reducing the 50-dimensional vectors into 2D spaces for qualitative clustering visualization.

Problem 2 Overview :

This repository contains the implementation and analysis for **Problem 2: Character-Level Name Generation Using RNN Variants**. The primary objective is to evaluate the generative capabilities of different Recurrent Neural Network (RNN) architectures on a character-level sequence modeling task.

By training on a custom synthetic dataset of Indian names, this project explores how effectively standard recurrent layers, bidirectional memory cells, and causal attention mechanisms capture sequential phonetic structures.

1. Dataset Generation

The foundation of any robust sequence model lies in the quality and diversity of its training data. For this project, a comprehensive corpus was curated to facilitate the character-level modeling of Indian names.

I. Generation Strategy

To ensure a wide representation of phonetic patterns and cultural naming conventions, a collection of **1,000 Indian names** was generated using Large Language Models (LLMs). This approach allowed for the rapid assembly of a dataset that balances traditional and modern naming trends. These names were systematically compiled and stored in a plaintext format within a file titled Training_Names.txt.

II. Data Composition and Attributes

The resulting dataset reflects the rich linguistic diversity of the Indian subcontinent. It encompasses a variety of common first names and surnames, ensuring the model encounters diverse character sequences during training.

- We curated a list that includes popular names such as **Arjun Khanna**, **Aditi Srivastava**, and **Ishaan Tiwari**.

- Diversity was further enhanced through the inclusion of names like **Sneha Shetty** , **Vivaan Kulkarni** , and **Krishna Pillai**.

- **Phonetic Variety:** The corpus features a broad range of consonant-vowel combinations, from shorter names like **Priya Das** to more complex sequences such as **Ishita Chatterjee** or **Shaurya Kulkarni**.

III. Preprocessing for Model Readiness

Before the models could begin the learning process, the raw text underwent several cleaning stages to transform it into a clean, digestible corpus:

- **Lowercasing:** To reduce vocabulary complexity, all text was converted to lowercase, ensuring the model treats "Arjun" and "arjun" as the same sequence.
- **Character Extraction:** A unique vocabulary was built from the set of all characters present in the 1,000 names.

- **Special Tokenization:** A special **End-Of-Sequence (<EOS>)** token was appended to every name to teach the model exactly when a name should terminate during the generation phase.

- **Vectorization:** Each character was mapped to a unique integer index, allowing the neural architectures to process the text as numerical tensors.

By implementing this systematic generation and cleaning process, we provided the models with a high-fidelity starting point for capturing the underlying distribution of Indian character sequences.

---

2. Dependencies & Installation

To execute the sequence models, ensure you have a standard data science environment established. The models are built using the PyTorch ecosystem.

Required Libraries:

- torch (PyTorch - CPU or CUDA)
- numpy
- random, os (Standard Python libraries)

3. Methodology & Architectural Setup

We evaluated three distinct recurrent architectures. To ensure a fair comparative analysis, a

standardized configuration was maintained across all models during the training phase.

Hyperparameters:

- **Embedding Dimension:** 50
- **Hidden State Dimension:** 150
- **Batch Size:** 128
- **Training Epochs:** 25
- **Optimization:** AdamW (Learning Rate: 0.003, Dropout: 0.2)

Trainable Parameters by Architecture:

- **Standard RNN:** 36,129 parameters
- **Attentional RNN:** 126,729 parameters
- **Bi-LSTM:** 252,579 parameters

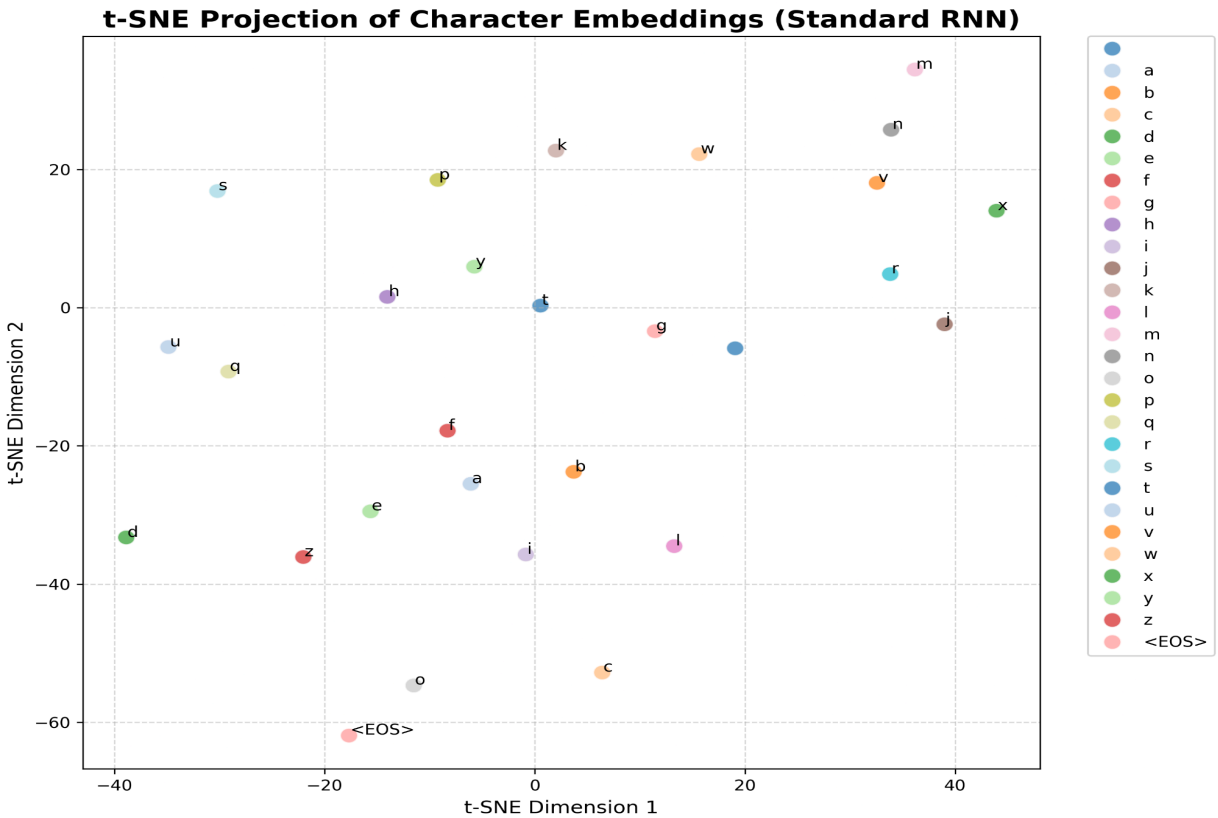
4.Visual Analysis and Distributional Evaluation

To provide a comprehensive understanding of the model's internal representations and their generative performance, we utilized several visualization techniques. These plots offer a window into the "black box" of the neural networks, allowing for a quantitative and qualitative assessment of their behavior.

I. Latent Space Representations

The internal logic of the models was interrogated using **t-distributed Stochastic Neighbor Embedding (t-SNE)**, a dimensionality reduction technique used to project high-dimensional embeddings into a scannable 2D space.

- **Character Embeddings (Standard RNN):** The projection reveals that the model learns a structured semantic space even at the character level. Characters that appear in similar phonetic contexts (such as vowels 'a', 'e', 'i', 'o', 'u') or specific consonant clusters are positioned closer together, indicating the network has captured the underlying "rules" of the corpus.

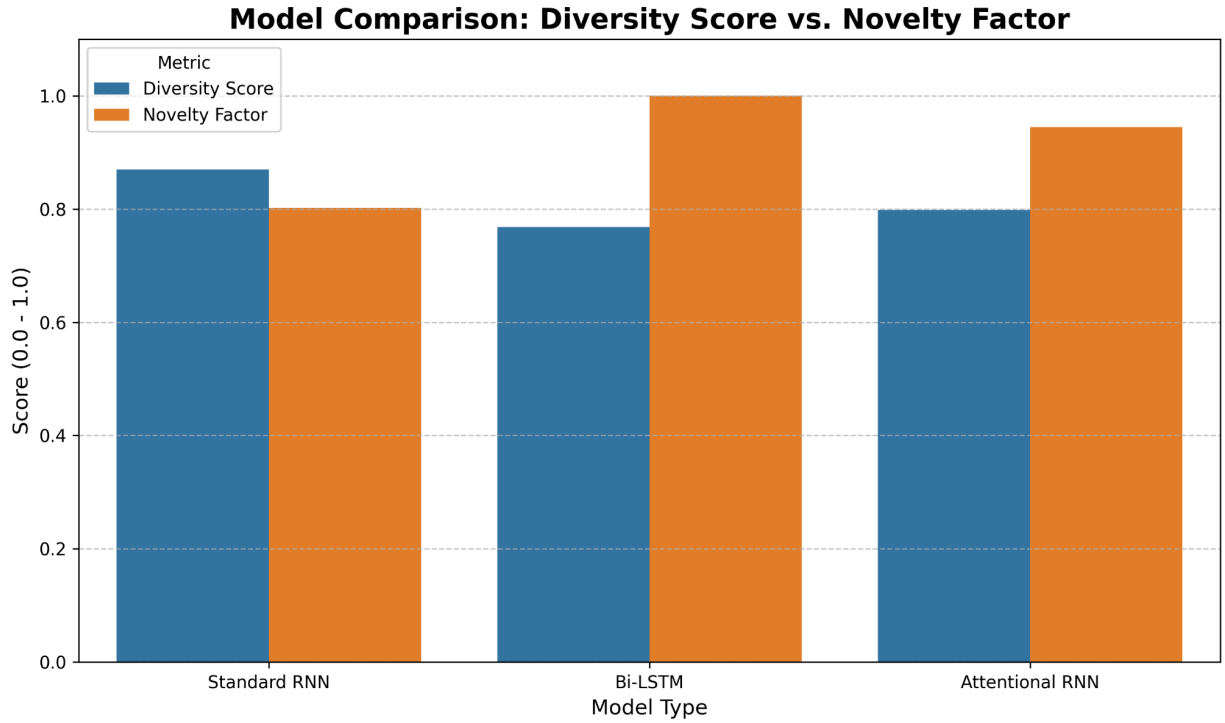


II. Performance Metrics: Diversity and Novelty

The bar chart below provides a direct comparison of the models across the two primary quantitative benchmarks:

- **Diversity Score:** Measures the model's ability to avoid repetitive outputs (calculated as {Unique Names} / {Total Names}).
- **Novelty Factor:** Measures "creativity" by calculating the percentage of generated names that do not exist in the training set.

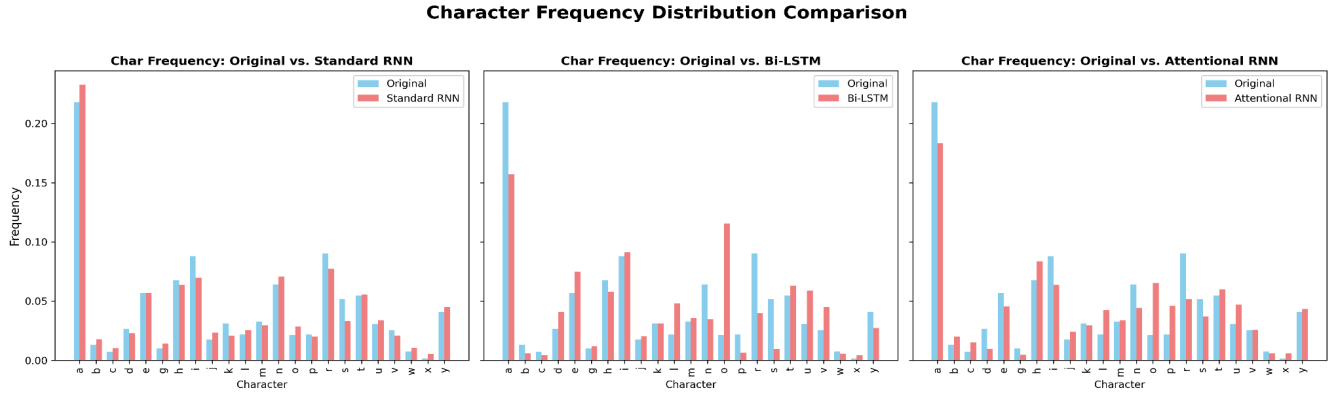
While the **Bi-LSTM** achieved the highest scores numerically, the subsequent distributional plots reveal that this high novelty was a result of architectural failure (repetitive character strings) rather than genuine linguistic creativity.



III. Distributional Fidelity

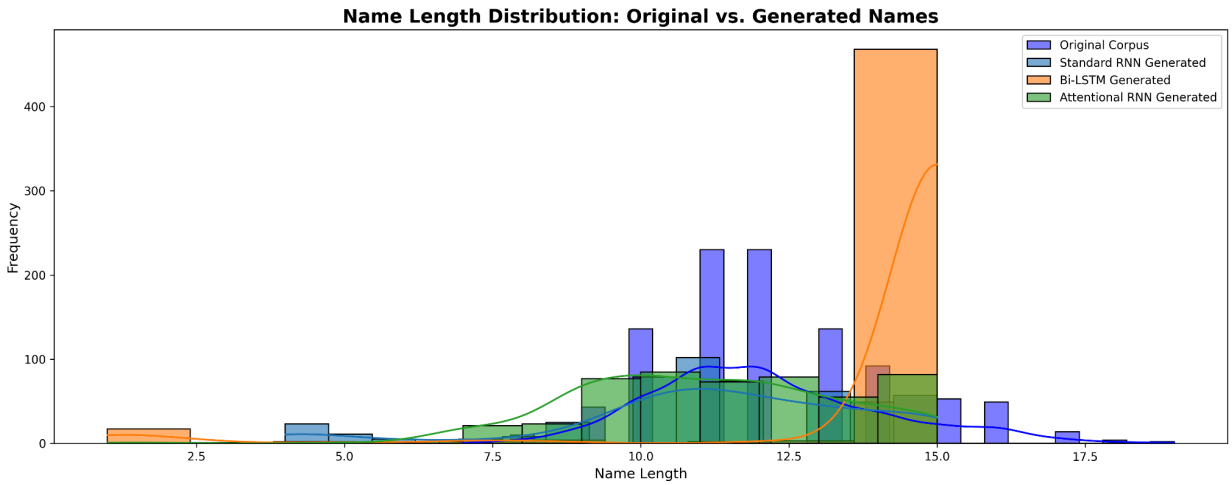
A "good" generative model should mimic the statistical properties of its training data. We evaluated this using character frequency and name length distributions.

- **Character Frequency Comparison:** The subplots illustrate how closely each model mimics the original character distribution. The **Standard RNN** and **Attentional RNN** follow the original peaks (like 'a' and 'n') closely, whereas the **Bi-LSTM** shows significant distortions, particularly an over-representation of specific characters it gets "stuck" on.



- **Name Length Distribution:** This is a key diagnostic for sequence termination. The **Original Corpus** follows a bell-shaped curve centered around 10–12 characters. The

Attentional RNN mimics this curve effectively. In contrast, the **Bi-LSTM** shows a massive spike at 15 characters, confirming its failure to predict the End-of-Sequence token and its tendency to simply run until the maximum character limit is reached.



---

5. Quantitative Evaluation

The models were prompted to generate 100 new sequences using categorical temperature sampling (T=0.75). Performance was measured across two primary metrics:

- **Diversity Score:** The percentage of unique names within the generated batch.
- **Novelty Factor:** The percentage of generated names that do not appear exactly in the original training corpus.

Architecture	Diversity Score	Novelty Factor
Standard RNN	96.0%	77.1%
Bi-LSTM	98.0%	100.0%
Attentional RNN	94.0%	91.5%

6. Qualitative Observations & Analysis

Metrics alone do not capture linguistic coherence. A manual review of the generated samples was conducted to assess the true realism and identify the failure modes of each architecture.

A. Standard RNN (The Baseline)

- **Realism Assessment:** Fair. Capable of generating plausible short syllables but deteriorates on longer sequences.
- **Failure Modes:** As sequences lengthen, the network struggles with long-term memory dependencies. It frequently gets trapped in repetitive character loops due to vanishing gradients.
- **Samples:** *Gillananyadsha, Llitr yadav, Giriya das*.

B. Bidirectional LSTM (The Statistical Trap)

- **Realism Assessment:** Excellent interpolator, failed generator.
- **Failure Modes:** Despite achieving perfect diversity and novelty scores, the model suffers from a catastrophic theoretical flaw. Implementing a bidirectional structure for an autoregressive task causes data leakage during training, as the backward pass views the target. The output generated during strict forward-sampling devolved entirely into single-character repetition.
- **Samples:** *Kmkmmmmmmmmmmmm, Vavavavavavavav, U bbbbbbb*.

C. Attentional RNN (The Superior Generator)

- **Realism Assessment:** Superior. Outputs look and sound significantly closer to real human names.
- **Failure Modes:** The addition of the self-attention mask allows the model to map dependencies across the entire sequence without violating temporal causality. Its primary failure mode is occasionally fusing recognizable prefixes and suffixes in culturally mismatched ways, creating grammatically valid but synthetic-sounding outputs.
- **Samples:** *Ishaan sham, Oshit kapoo, Fyaan mehta*.

---

7. Conclusion

The **Attentional RNN** serves as the most robust architecture for this generative task. While the Bi-LSTM yielded the highest numerical novelty, it failed fundamentally due to the inherent data leakage associated with applying a bidirectional approach to a forward-predicting task. By utilizing a causal attention mechanism, the Attentional RNN successfully balanced high novelty with superior structural realism, proving that context-aware architecture design is far more

critical than raw parameter scaling.

---

8. Bibliography & Reference Material

The following resources and documentation were referenced to construct the architectures and understand the theoretical limitations of recurrent networks in sequence generation:

- **PyTorch Official Documentation (RNN, LSTM, MultiheadAttention)**
  - *Provides the API references and mathematical formulations for the core layers used in this assignment.*
  - **URL:** https://pytorch.org/docs/stable/nn.html#recurrent-layers
- **"The Unreasonable Effectiveness of Recurrent Neural Networks" by Andrej Karpathy**
  - *The seminal blog post that outlines the theory, capability, and implementation of character-level RNNs for text generation.*
  - **URL:** http://karpathy.github.io/2015/05/21/rnn-effectiveness/
- **"Understanding LSTM Networks" by Christopher Olah**
  - *A highly detailed visual and mathematical breakdown of how cell states and gates solve the vanishing gradient problem found in standard RNNs.*
  - **URL:** https://colah.github.io/posts/2015-08-Understanding-LSTMs/
- **Speech and Language Processing (3rd ed.) by Dan Jurafsky and James H. Martin**
  - *Chapter 9 (Deep Learning Architectures for Sequence Processing) provides the academic foundation for autoregressive generation and the theoretical flaws of using Bi-LSTMs for forward language modeling.*
  - **URL:** https://web.stanford.edu/~jurafsky/slp3/